

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет
по учебной (ознакомительной) практике

Студент:
гр. 558301 Ермаков В. С.

Руководитель:
старший преподаватель
Ковальчук А. М.

МИНСК 2026

СОДЕРЖАНИЕ

Введение	3
1 Проектирование и разработка	4
1.1 Проектирование структур данных	4
1.2 Алгоритм функции <code>tree_add_element</code>	6
1.3 Алгоритм функции <code>tree_save_direct</code>	6
2 Тестирование и отладка	8
2.1 Результаты работы программы	8
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12
ПРИЛОЖЕНИЕ А Схема алгоритма функции <code>main</code>	13
ПРИЛОЖЕНИЕ Б Схема алгоритма функции <code>tree_add_element</code>	15
ПРИЛОЖЕНИЕ В Код программы	17
ПРИЛОЖЕНИЕ Г Ведомость документов	35

ВВЕДЕНИЕ

Автоматизация процессов купли-продажи и создание удобных каталогов товаров является одной из основных задач современной электронной коммерции. Для обеспечения высокой производительности и оптимального использования системных ресурсов такие программные продукты требуют тщательного проектирования структур данных и алгоритмов обработки информации.

Целью данной работы является разработка консольного приложения на языке программирования С, реализующего базовый функционал маркетплейса с гибкой системой категорий.

Для достижения поставленной цели необходимо решить следующие задачи:

- спроектировать масштабируемые структуры данных для хранения динамических характеристик товаров;
- разработать систему сохранения и загрузки каталога товаров с использованием файлов;
- реализовать консольный пользовательский интерфейс с отображением товаров;
- реализовать логику навигации и поиска по каталогу, добавления товаров и категорий.

1 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА

1.1 Проектирование структур данных

В таблице 1.1 представлена иерархия структур данных, описывающих товары. Структуры расположены сверху и объединяют свои внутренние поля, находящиеся на уровнях ниже. Для размещения широкой таблицы на странице применен физический поворот всей фигуры вместе с заголовком. Независимые структуры и перечисления вынесены в таблицы 1.2 и 1.3.

Таблица 1.1 — Иерархия и состав структур данных каталога товаров

struct TreeNode									
		struct Product item							
TreeNode* left Левый потомок	TreeNode* right Правый потомок	int id Иденти- фикатор товара	char name[30] Название	char desc[50] Описание	int cost Цена	int rating Рейтинг	struct Characteristics cs		
							int category_id Иденти- фикатор категории	union Field fields[6]	
								int number Целое	float real Дробное char str[30] Строка

Таблица 1.2 — Структура данных категории

struct Category			
char name[30] Название	int fields_count Кол-во полей	Type field_types[6] Типы полей	char field_names[6][30] Имена полей

Таблица 1.3 — Перечисление типов данных характеристик

enum Type		
INT (0) Целое	FLOAT (1) Дробное	STR (2) Строка

1.2 Алгоритм функции tree_add_element

Функция добавления нового товара в бинарное дерево поиска игнорируя дубликаты по идентификатору.

Вх: current – адрес адреса текущего узла дерева, тип TreeNode**
x – структура данных товара, тип Product.

- 1 Начало.
- 2 Цикл по элементам дерева пока адрес *current не равен NULL.
- 3 Если x.id равно (*current)->item.id, перейти к шагу 7.
- 4 Присвоить переменной current результат выполнения функции tree_get_next для перехода к следующей вершине.
- 5 Конец цикла по элементам дерева.
- 6 Присвоить переменной *current результат выполнения функции tree_create_node для выделения памяти под новую вершину.
- 7 Конец.

1.3 Алгоритм функции tree_save_direct

Функция рекурсивно выполняет прямой обход бинарного дерева поиска и сохраняет данные каждого товара в бинарный файл.

Вх: current – адрес текущего узла дерева, тип TreeNode*
file – указатель на открытый файловый поток, тип FILE*.

- 1 Начало.
- 2 Если текущая вершина current равна NULL, перейти к шагу 6.
- 3 Вызвать функцию fwrite для записи товара current->item в бинарный файл.

- 4 Вызвать функцию `tree_save_direct` для рекурсивной записи в файл левого поддерева.
- 5 Вызвать функцию `tree_save_direct` для рекурсивной записи в файл правого поддерева.
- 6 Конец.

2 ТЕСТИРОВАНИЕ И ОТЛАДКА

2.1 Результаты работы программы

Тестирование разработанного программного обеспечения проводилось методом функционального тестирования. В ходе тестирования проверялась корректность работы алгоритмов считывания, добавления, фильтрации и сохранения данных.

На рисунке 2.1 представлено главное меню разработанного консольного приложения.

```
i - Import products
c - Add category
a - Add product
x - Exit
Enter action (i/c/a/x):
```

Рисунок 2.1 — Главное меню программы

Процесс динамического формирования новой категории товаров с заданием пользовательских полей и их типов данных продемонстрирован на рисунке 2.2.

```
Category name: Electronics
Number of additional fields (up to 6): 3
Field 1 name: Brand
Field type (0 - Integer, 1 - Float, 2 - String): 2
Field 2 name: Warranty (months)
Field type (0 - Integer, 1 - Float, 2 - String): 0
Field 3 name: Weight (kg)
Field type (0 - Integer, 1 - Float, 2 - String): 1
All data correct? Enter action (y/n): t
Enter action (y/n): y
Category successfully added!

Press Esc to return to menu.
```

Рисунок 2.2 — Добавление новой категории товаров

Ввод информации о новом товаре на основе ранее сформированной структуры категории показан на рисунке 2.3. Программа успешно запрашивает специфичные поля.

```
Available categories:
0 - Electronics
1 - Books
2 - Clothing
3 - Metal

Name: GPhone 42 Pro Max Ultra
Description: The worst phone ever
Cost: 4242
Rating (from 1 to 5): 1
Category ID: 0
Brand: Gapple
Warranty (months): 0
Weight (kg): 0.42

ID: 4
Name: GPhone 42 Pro Max Ultra
Category: Electronics
Cost: 4242
Rating: *

Description:
The worst phone ever

Brand: Gapple
Warranty (months): 0
Weight (kg): 0.42
All data correct? Enter action (y/n): y
Product added!

Press Esc to return to menu.
```

Рисунок 2.3 — Добавление нового товара в каталог

На рисунке 2.4 представлен вывод каталога товаров в виде форматированной таблицы с применением фильтрации по категории и сортировки на основе бинарного дерева поиска.

```
Sorting by rating
a - Ascending
d - Descending
Enter action (a/d): a
Filter by category? Enter action (y/n): n
#  Name                      Category      Cost  Rating
0  Book "Clean code"         Books         999   *****
1  Mac macro 1 PB            Electronics   1500   ****
2  BSUIR T-Shirt             Clothing      15     ****
3  GPhone 42 Pro Max Ultra    Electronics   4242   *
Press Enter to view a product, Esc to return to menu
```

Рисунок 2.4 — Просмотр каталога товаров

Детальная информация о выбранном товаре (карточка товара), включая его цветной рейтинг и дополнительные характеристики, представлена на рисунке 2.5.

```
ID: 0
Name: GPhone 42 Pro Max Ultra
Category: Electronics
Cost: 4242
Rating: *

Description:
Worth smartphone ever

Brand: Gapple
Warranty (months): 0
Weight (kg): 4.20

Press Enter to rate the product, Esc for product list.
```

Рисунок 2.5 — Детальный просмотр карточки товара

ЗАКЛЮЧЕНИЕ

В ходе ознакомительной практики разработано консольное приложение на языке С для управления каталогом товаров.

В рамках индивидуального задания решены следующие задачи:

- спроектированы структуры данных для описания категорий и товаров с поддержкой настраиваемых характеристик;
- реализованы алгоритмы добавления и поиска элементов на основе бинарного дерева поиска;
- организован файловый ввод-вывод для сохранения состояния каталога;
- создан консольный интерфейс для взаимодействия с пользователем.

В результате выполнения работы закреплены практические навыки использования динамической памяти, указателей и работы с файлами в С. Разработанная программа успешно отлажена.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] СТП 01–2024. Дипломные проекты (работы). Общие требования. – Минск : БГУИР, 2024. – 122 с.

[2] Основы алгоритмизации и программирования : лаб. практикум для студ. спец. 1-40 02 01 «Вычислительные машины, системы и сети» всех форм обуч. В 2 ч. Ч. 2 / сост. Ю. А. Луцик [и др.]. – Минск : БГУИР, 2010. – 36 с. : ил.

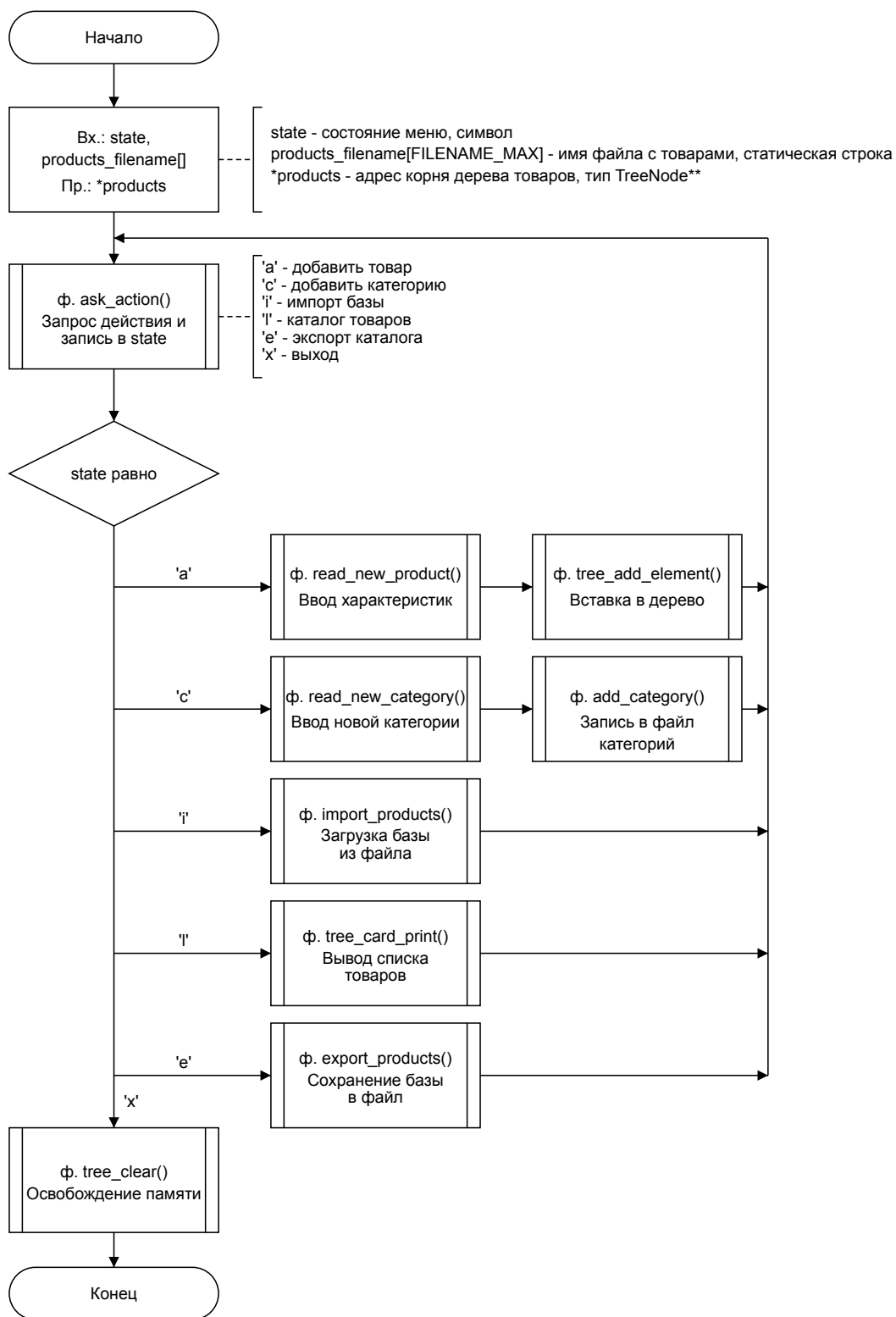
[3] Бхаргава, А. Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих / А. Бхаргава. – СПб. : Питер, 2017. – 288 с.

[4] Справочник по языку C [Электронный ресурс]. – Режим доступа: <https://ru.cppreference.com/w/c>. – Дата доступа: 15.05.2024.

[5] Документация Microsoft C/C++ и языка C [Электронный ресурс] / Microsoft Learn. – Режим доступа: <https://learn.microsoft.com/ru-ru/cpp/c-language/>. – Дата доступа: 16.05.2024.

ПРИЛОЖЕНИЕ А
(Обязательное)

Схема алгоритма функции main

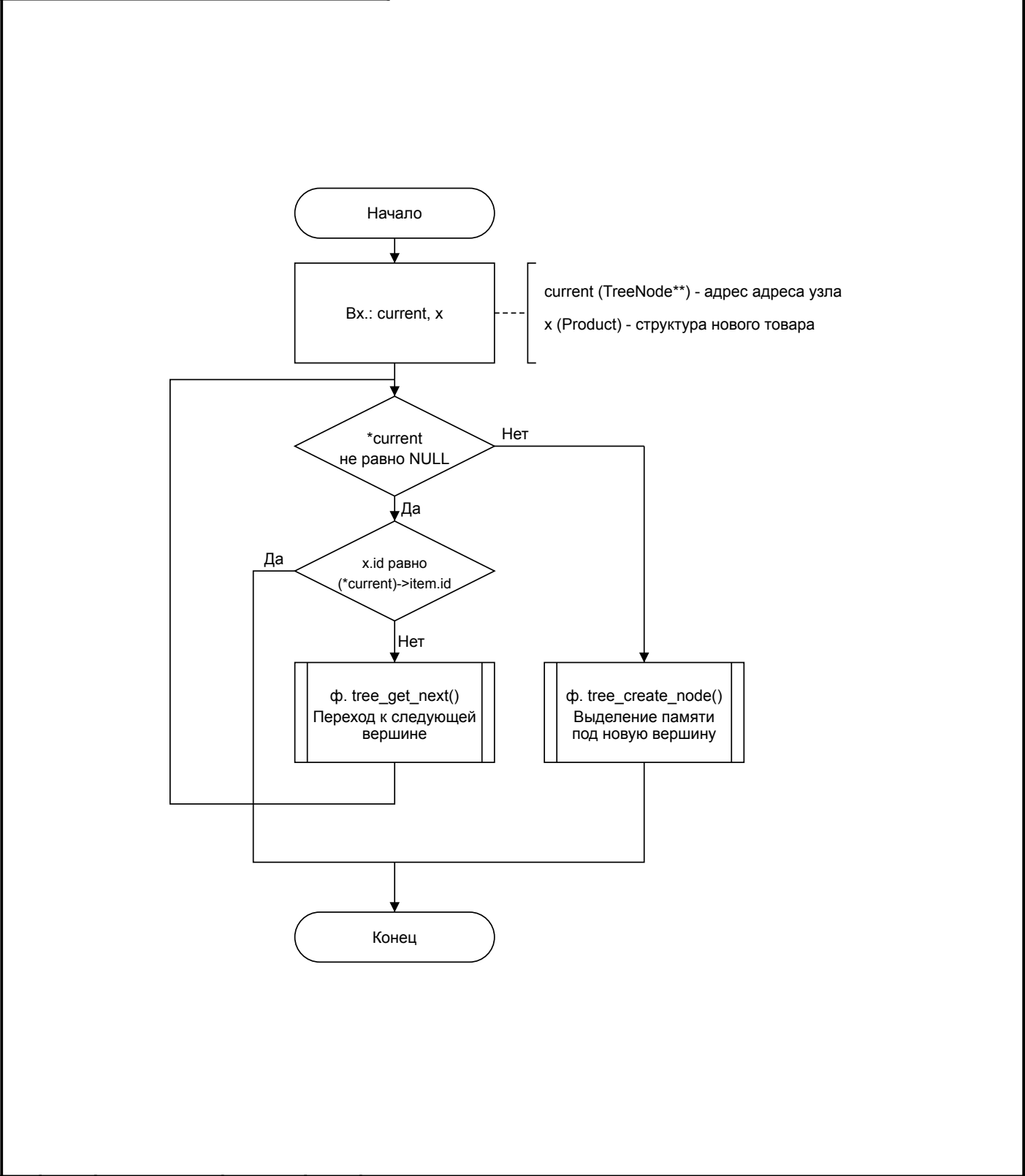


					ГЧИР.6-05-0611-05.114 ПД1									
					Схема алгоритма функции main					Лист.		Масса	Масштаб	
Изм.	Лист	№ докум.	Подпись	Дата							Т			
Разраб.		Ермаков												
Проб.		Ковальчук												
										Лист		Листов 1		
										ЭВМ, зр. 558301				

ПРИЛОЖЕНИЕ Б

(Обязательное)

Схема алгоритма функции `tree_add_element`



					ГЧИР.6-05-0611-05.114 ПД2						
					Схема алгоритма функции tree_add_element	Лист.		Масса		Масштаб	
Изм.	Лист	№ докум.	Подпись	Дата			Т				
Разраб.		Ермаков									
Проб.		Кобальчук									
						Лист		Листов 1			
						ЭВМ, зр. 558301					

ПРИЛОЖЕНИЕ В

(Обязательное)

Код программы

Код файла main.c

```
#include <locale.h>
#include <string.h>
#include "categories.h"
#include "interaction.h"
#include "io.h"
#include "product.h"
#include "stack.h"
#include "tree.h"

int main(void)
{
    char products_filename[FILENAME_MAX];
    TreeNode* products = NULL, *current_product = NULL;

    State state = MENU;
    do
    {
        clear_screen();

        State prev_state = state;
        switch (state)
        {
            case MENU:
                printf("i - Import products\n");
                printf("c - Add category\n");
                if (get_categories_count() > 0)
                    printf("a - Add product\n");
                if (products != NULL)
                {
                    printf("l - Product catalog\n");
                    printf("e - Export products\n");
                }
                printf("x - Exit\n");

                state = ask_action(get_categories_count() > 0 ?
(products != NULL ? "icalxe" : "icax") : "icx");
                break;

            case IMPORT_PRODUCTS:
                printf("Enter filename to import products (Enter for %s):", PRODUCTS_DEFAULT_FILENAME);
                read_string_or_default(products_filename,
PRODUCTS_DEFAULT_FILENAME);

                int prev_products_count = tree_get_size(products, -1);
```

```

import_products(&products, products_filename);
int current_products_count = tree_get_size(products, -1);

printf(current_products_count - prev_products_count > 0 ?
SET_GREEN : SET_RED);
printf("Imported %d products, total products %d\n" RESET,
current_products_count - prev_products_count, current_products_count);
break;

case ADD_PRODUCT:
    Product product;
    do
    {
        print_available_categories();
        product = read_new_product(tree_get_size(products,
-1));

        printf("\n");
        print_product_details(product);
        printf("All data correct? ");
    } while (ask_action("yn") == 'n');
    tree_add_element(&products, product);
    printf(SET_GREEN "Product added!\n" RESET);
    break;

case ADD_CATEGORY:
    Category category;
    do {
        printf("\n");
        category = read_new_category();
        printf("All data correct? ");
    } while (ask_action("yn") == 'n');
    add_category(category);
    printf(SET_GREEN "Category successfully added!\n" RESET);
    break;

case EXPORT_PRODUCTS:
    printf("Enter filename to export products (Enter for %s):
", PRODUCTS_DEFAULT_FILENAME);
    read_string_or_default(products_filename,
PRODUCTS_DEFAULT_FILENAME);
    export_products(products, products_filename);
    printf(SET_GREEN "Products saved to file!\n" RESET);
    break;

case LIST_PRODUCTS:
    printf("Sorting by rating\n");
    printf("a - Ascending\n");
    printf("d - Descending\n");
    bool ascending = (ask_action("ad") == 'a');

    printf("Filter by category? ");
    int filter = -1;
    if (ask_action("yn") == 'y')

```

```

    {
        print_available_categories();
        printf("Enter category ID: ");
        filter = read_int_range(0, get_categories_count() -
1);
    }

    print_products_header();
    tree_card_print(products, ascending, filter, 0);

    printf("Press Enter to view a product, Esc to return to
menu\n");
    if (wait_special_symbol(true, true) == ENTER)
    {
        printf("Enter the product index to view.\n");
        current_product = tree_find(products,
read_int_range(0, tree_get_size(products, filter) - 1), filter,
ascending);
        state = VIEW_PRODUCT;
    }
    else
        state = MENU;

    break;

case VIEW_PRODUCT:
    print_product_details(current_product->item);

    printf("\nPress Enter to rate the product, Esc for product
list.\n");

    if (wait_special_symbol(true, true) == ENTER)
    {
        printf("Rate the product from 1 to %d: ", MAX_RATING);
        int new_rating = read_int_range(1, MAX_RATING);

        if (new_rating != current_product->item.rating)
        {
            Product updated_product = current_product->item;
            tree_remove_element(&products, updated_product);
            updated_product.rating = new_rating;
            tree_add_element(&products, updated_product);
        }
    }

    state = LIST_PRODUCTS;
    break;
}

if (prev_state != MENU && prev_state != LIST_PRODUCTS)
{
    printf("\nPress Esc to return to menu.");
    wait_special_symbol(true, false);
}

```

```

        state = MENU;
    }
} while (state != 'x');

tree_clear(&products);

return 0;
}

```

Код файла product.h

```

#pragma once
#include "settings.h"

typedef struct TreeNode TreeNode;

union Field
{
    int number;
    float real;
    char str[MAX_LENGTH];
};
typedef union Field Field;

struct Characteristics
{
    int category_id;
    Field fields[MAX_FIELD_COUNT];
};
typedef struct Characteristics Characteristics;

struct Product
{
    int id;
    char name[MAX_LENGTH];
    char description[DESCRIPTION_LENGTH];
    int cost;
    int rating;
    Characteristics cs;
};
typedef struct Product Product;

void import_products(TreeNode** products, char* filename); // Импорт
товаров из файла
void export_products(TreeNode* products, char* filename); // Экспорт
товаров в файл
Product read_new_product(int new_id); // Ввод данных нового товара

void print_rating(int rating); // Вывод рейтинга товара

void print_products_header(); // Вывод шапки таблицы товаров
void print_product_row(Product product, int index); // Вывод одного
товара в виде строки таблицы

```

```
void print_product_details(Product product); // Вывод большой карточки товара
```

Код файла categories.h

```
#pragma once
#include "settings.h"

typedef struct Characteristics Characteristics;

enum Type
{
    INT,
    FLOAT,
    STR,
};
typedef enum Type Type;

struct Category
{
    char name[MAX_LENGTH];
    int fields_count;
    Type field_types[MAX_FIELD_COUNT];
    char field_names[MAX_FIELD_COUNT][MAX_LENGTH];
};
typedef struct Category Category;

Category read_new_category(); // Ввод новой категории
int get_categories_count(); // Получение количества категорий

void add_category(Category category); // Добавление категории в файл
Category get_category(int index); // Получение категории по индексу

void print_available_categories(); // Вывод списка доступных категорий
void print_category_field(Category category, Characteristics cs, int i); // Вывод поля категории
```

Код файла tree.h

```
#pragma once
#include <malloc.h>
#include <stdio.h>
#include <stdbool.h>
#include "product.h"

typedef struct TreeNode TreeNode;
struct TreeNode
{
    Product item;
    TreeNode *left, *right;
};

TreeNode* tree_create_node(Product x); // Создать узел

int childs_count(TreeNode* node); // Получение количества детей узла
```

```

int tree_get_depth(TreeNode* node); // Узнать глубину дерева

TreeNode** tree_get_next(TreeNode* node, Product x); // Поиск
следующей вершины для поиска
TreeNode** tree_find_smallest(TreeNode** node); // Найти наименьший
узел в поддереве

void tree_add_element(TreeNode** current, Product x); // Добавить
элемент в дерево
bool tree_remove_element(TreeNode** current, Product x); // Удалить
элемент из дерева
void tree_clear(TreeNode** node); // Очистить дерево

TreeNode* tree_find(TreeNode* node, int index, int category_filter,
bool ascending); // Поиск товара в дереве по индексу

int tree_get_size(TreeNode* node, int category_filter); // Получение
количества элементов в дереве
void tree_save_direct(TreeNode* current, FILE* file); // Сохранение
дерева в файл (прямой обход)
void tree_card_print(TreeNode* node, bool ascending, int
category_filter, int index_offset); // Вывод товаров (симметричный
обход)

```

Код файла interaction.h

```

#pragma once
#include <stdbool.h>

enum State {
    MENU = 'm',
    ADD_PRODUCT = 'a',
    ADD_CATEGORY = 'c',
    IMPORT_PRODUCTS = 'i',
    EXPORT_PRODUCTS = 'e',
    LIST_PRODUCTS = 'l',
    VIEW_PRODUCT = 'v',
};
typedef enum State State;

char ask_action(const char* acceptable_chars); // Спрашивает
пользователя о текущем действии
void read_string_or_default(char* out, const char* default_string); //
Ввод строки или установка значения по умолчанию
char wait_special_symbol(bool esc, bool enter); // Ожидание ввода
специального символа

```

Код файла io.h

```

#pragma once
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

#define SET_RED "\x1B[31m"

```

```

#define SET_GREEN "\x1B[32m"
#define SET_YELLOW "\x1B[33m"
#define SET_BLUE "\x1B[34m"
#define RESET "\x1B[0m"

#define ESC '\033'
#define ESC_R '\177'
#define ENTER '\n'
#define ALL_SPECIAL_CHARS "\033\r"

FILE* open_file(char* filename, bool binary, bool write_mode, bool
exit_on_fail); // Открытие файла с обработкой ошибок
int read_int_range(int from, int to); // Ввод целого числа в заданном
диапазоне
int read_int(FILE* file, bool binary); // Ввод целого числа
float read_float(FILE* file, bool binary); // Ввод вещественного числа
void read_string(char* str, FILE* file, bool until_ln); // Ввод строки
void clear_screen(); // Очистка экрана
void flush_buffer(FILE* file, bool stdin_skip); // Очистка буфера
ВВОДА
void print_n_times(int n, char c); // Вывод символа n раз

```

Код файла settings.h

```

#pragma once

#define CATEGORIES_FILENAME "categories.dat"
#define PRODUCTS_DEFAULT_FILENAME "products.dat"

#define MAX_LENGTH 30
#define DESCRIPTION_LENGTH 50

#define MAX_FIELD_COUNT 6

#define MAX_RATING 5

```

Код файла product.c

```

#include "product.h"
#include "io.h"
#include "stack.h"
#include "tree.h"
#include "categories.h"
#include <string.h>

// Импорт товаров из файла
void import_products(TreeNode** products, char* filename)
{
    FILE* file = open_file(filename, true, false, false);
    if (file == NULL)
        return;

    while (!feof(file))
    {
        Product product;
    }
}

```

```

    fread(&product, sizeof(Product), 1, file);
    tree_add_element(products, product);
}

fclose(file);
}

// Экспорт товаров в файл
void export_products(TreeNode* products, char* filename)
{
    FILE* file = open_file(filename, true, true, false);
    if (file == NULL)
    {
        printf("Невозможно записать в файл %s", filename);
        return;
    }

    tree_save_direct(products, file);

    fclose(file);
}

// Ввод данных нового товара
Product read_new_product(int new_id)
{
    Product product;
    product.id = new_id;
    printf("Name: ");
    read_string(product.name, stdin, true);
    printf("Description: ");
    read_string(product.description, stdin, true);
    printf("Cost: ");
    product.cost = read_int(stdin, false);
    printf("Rating (from 1 to %d): ", MAX_RATING);
    product.rating = read_int_range(1, MAX_RATING);
    printf("Category ID: ");
    product.cs.category_id = read_int(stdin, false);

    Category cat = get_category(product.cs.category_id);
    for (int i = 0; i < cat.fields_count; i++)
    {
        printf("%s: ", cat.field_names[i]);

        if (cat.field_types[i] == INT)
            product.cs.fields[i].number = read_int(stdin, false);

        else if (cat.field_types[i] == FLOAT)
            product.cs.fields[i].real = read_float(stdin, false);

        else if (cat.field_types[i] == STR)
            read_string(product.cs.fields[i].str, stdin, true);
    }
}

```



```

    return product;
}

// Вывод рейтинга товара
void print_rating(int rating)
{
    if (rating <= 2) printf(SET_RED);
    else if (rating == 3) printf(SET_YELLOW);
    else printf(SET_GREEN);

    for (int i = 0; i < rating; i++) printf("*");
    for (int i = rating; i < MAX_RATING; i++) printf(" ");

    printf(RESET);
}

// Вывод шапки таблицы товаров
void print_products_header()
{
    printf(SET_BLUE);
    printf("%-2s %-*s %-*s %-5s %-10s \n",
        "#", MAX_LENGTH, "Name", MAX_LENGTH, "Category", "Cost",
        "Rating");
    printf(RESET);
}

// Вывод одного товара в виде строки таблицы
void print_product_row(Product product, int index)
{
    Category category = get_category(product.cs.category_id);

    printf("%-2d %-*s %-*s %-5d ",
        index, MAX_LENGTH, product.name, MAX_LENGTH, category.name,
        product.cost);

    print_rating(product.rating);
    printf("\n");
}

// Вывод большой карточки товара
void print_product_details(Product product)
{
    Category category = get_category(product.cs.category_id);

    printf("ID: %d\n", product.id);
    printf("Name: %s\n", product.name);
    printf("Category: %s\n", category.name);
    printf("Cost: %d\n", product.cost);
    printf("Rating: ");
    print_rating(product.rating);
    printf("\n");

    printf("\nDescription: %s\n\n", product.description);
}

```

```

    for (int i = 0; i < category.fields_count; i++)
    {
        print_category_field(category, product.cs, i);
        printf("\n");
    }
}

```

Код файла categories.c

```

#include "categories.h"
#include "product.h"
#include "io.h"

// Получение количества категорий
int get_categories_count()
{
    FILE* file = fopen(CATEGORIES_FILENAME, "rb");
    if (file == NULL)
        return 0;

    fseek(file, 0, SEEK_END);
    int count = ftell(file) / sizeof(Category);
    fclose(file);

    return count;
}

// Добавление категории в файл
void add_category(Category category)
{
    FILE* file = fopen(CATEGORIES_FILENAME, "ab");
    if (file == NULL)
    {
        printf("Failed to open file %s to add category.\n",
CATEGORIES_FILENAME);
        exit(0);
    }

    fwrite(&category, sizeof(Category), 1, file);
    fclose(file);
}

// Получение категории по индексу
Category get_category(int index)
{
    Category category;
    FILE* file = open_file(CATEGORIES_FILENAME, true, false, true);
    fseek(file, index * sizeof(Category), SEEK_SET);
    fread(&category, sizeof(Category), 1, file);
    fclose(file);
    return category;
}

```

```

// Вывод списка доступных категорий
void print_available_categories()
{
    int categories_count = get_categories_count();
    printf("\nAvailable categories:\n");
    for (int i = 0; i < categories_count; i++)
        printf("%d - %s\n", i, get_category(i).name);
    printf("\n");
}

// Ввод новой категории
Category read_new_category()
{
    Category category;
    printf("Category name: ");
    read_string(category.name, stdin, true);

    printf("Number of additional fields (up to %d): ",
MAX_FIELD_COUNT);
    category.fields_count = read_int_range(0, MAX_FIELD_COUNT);

    for (int i = 0; i < category.fields_count; i++)
    {
        printf("Field %d name: ", i + 1);
        read_string(category.field_names[i], stdin, true);
        printf("Field type (0 - Integer, 1 - Float, 2 - String): ");
        category.field_types[i] = (Type)read_int_range(0, 2);
    }
    return category;
}

// Вывод поля категории
void print_category_field(Category category, Characteristics cs, int
i)
{
    if (category.field_types[i] == INT)
        printf("%s: %d", category.field_names[i],
cs.fields[i].number);

    if (category.field_types[i] == STR)
        printf("%s: %s", category.field_names[i], cs.fields[i].str);

    if (category.field_types[i] == FLOAT)
        printf("%s: %.2f", category.field_names[i],
cs.fields[i].real);
}

```

Код файла tree.c

```

#include "tree.h"
#include "utils.h"
#include <assert.h>
#include <stdio.h>
#include <stdbool.h>

```

```

// Создать узел
TreeNode* tree_create_node(Product x)
{
    TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));
    node->item = x;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// Получение количества детей узла
int childs_count(TreeNode* node)
{
    if (node == NULL)
        return 0;

    return (int)(node->left != NULL) + (int)(node->right != NULL);
}

// Узнать глубину дерева
int tree_get_depth(TreeNode* node)
{
    if (node == NULL)
        return 0;

    return max(tree_get_depth(node->left), tree_get_depth(node->right))
+ 1;
}

// Поиск следующей вершины для поиска
TreeNode** tree_get_next(TreeNode* node, Product x)
{
    return (x.rating <= node->item.rating) ? (&node->left) : (&node->right);
}

// Найти наименьший узел в поддереве
TreeNode** tree_find_smallest(TreeNode** node)
{
    assert((*node) != NULL);

    while ((*node)->left != NULL)
        node = &(*node)->left;

    return node;
}

// Добавить элемент в дерево
void tree_add_element(TreeNode** current, Product x)
{
    while ((*current) != NULL)
    {

```

```

    if (x.id == (*current)->item.id)
        return;
    current = tree_get_next(*current, x);
}

(*current) = tree_create_node(x);
}

// Удалить элемент из дерева
bool tree_remove_element(TreeNode** current, Product x)
{
    while ((*current) != NULL && (*current)->item.id != x.id) // Поиск
        // элемента равного x
        current = tree_get_next(*current, x);

    if ((*current) == NULL)
        return false;

    TreeNode* to_remove = (*current);

    if (childs_count(to_remove) == 2) // Если у вершины 2 детей
    {
        // Поиск наименьшего элемента в правом поддереве (первый больший)
        TreeNode** smallest_right = tree_find_smallest(&to_remove->right);
        TreeNode* instead_of_removed = *smallest_right;

        // Вставляем первый больший вместо удалённого
        (*smallest_right) = instead_of_removed->right;

        instead_of_removed->left = to_remove->left;
        instead_of_removed->right = to_remove->right;

        (*current) = instead_of_removed;
    }
    else // Если у вершины 0 и 1 ребёнок
        (*current) = (to_remove->left != NULL ? to_remove->left :
to_remove->right);

    free(to_remove);

    return true;
}

// Очистить дерево
void tree_clear(TreeNode** node)
{
    if ((*node) == NULL)
        return;

    tree_clear(&((*node)->left));
    tree_clear(&((*node)->right));

    free(*node);
}

```

```

    (*node) = NULL;
}

// Поиск товара в дереве по индексу
TreeNode* tree_find(TreeNode* node, int index, int category_filter,
bool ascending)
{
    if (node == NULL)
        return NULL;

    TreeNode* first = (ascending ? node->right : node->left);
    TreeNode* second = (ascending ? node->left : node->right);

    int first_size = tree_get_size(first, category_filter);
    bool this_node_correct = (category_filter == -1 || node-
>item.cs.category_id == category_filter);

    if (this_node_correct && index == first_size)
        return node;

    if (index < first_size)
        return tree_find(first, index, category_filter, ascending);

    return tree_find(second, index - first_size - this_node_correct,
category_filter, ascending);
}

// Получение количества элементов в дереве (с учетом фильтра)
int tree_get_size(TreeNode* node, int category_filter)
{
    if (node == NULL)
        return 0;

    bool this_node_correct = (category_filter == -1 || node-
>item.cs.category_id == category_filter);
    return tree_get_size(node->left, category_filter) +
tree_get_size(node->right, category_filter) + this_node_correct;
}

// Сохранение дерева в файл (прямой обход)
void tree_save_direct(TreeNode* current, FILE* file)
{
    if (current == NULL) return;
    fwrite(&current->item, sizeof(Product), 1, file);
    tree_save_direct(current->left, file);
    tree_save_direct(current->right, file);
}

// Вывод товаров (симметричный обход)
void tree_card_print(TreeNode* node, bool ascending, int
category_filter, int index_offset)
{
    if (node == NULL)

```

```

    return;

    TreeNode* first = (ascending ? node->right : node->left);
    TreeNode* second = (ascending ? node->left : node->right);

    int current_index = index_offset + tree_get_size(first,
category_filter);

    tree_card_print(first, ascending, category_filter, index_offset);
    if (category_filter == -1 || node->item.cs.category_id ==
category_filter)
        print_product_row(node->item, current_index);
    tree_card_print(second, ascending, category_filter, current_index +
1);
}

```

Код файла interaction.c

```

#include "interaction.h"
#include "io.h"
#include <string.h>
#include <conio.h>

#include "settings.h"

// Вывод доступных символов для ввода
void print_ask_chars(const char* acceptable_chars)
{
    printf("Enter action (");
    int n = strlen(acceptable_chars);
    for (int i = 0; i < n - 1; i++)
        printf("%c/", acceptable_chars[i]);
    printf("%c", acceptable_chars[n - 1]);
    printf("): ");
}

// Спрашивает пользователя о текущем действии
char ask_action(const char* acceptable_chars)
{
    print_ask_chars(acceptable_chars);

    char answer;
    bool valid_char = false;
    do
    {
        flush_buffer(stdin, true);
        answer = _getch();
        printf("%c\n", answer);
        valid_char = (strchr(acceptable_chars, answer) == NULL);
        if (valid_char)
            print_ask_chars(acceptable_chars);
    }
    while (valid_char);
}

```

```

        return answer;
    }

    // Ожидание ввода специального символа
    char wait_special_symbol(bool esc, bool enter)
    {
        char answer;
        do
        {
            flush_buffer(stdin, true);
            answer = _getch();
        } while (!(esc && answer == ESC) || (esc && answer == ESC_R) ||
            (enter && answer == ENTER));
        return answer;
    }

    // Ввод строки или установка значения по умолчанию
    void read_string_or_default(char* out, const char* default_string)
    {
        flush_buffer(stdin, true);
        fgets(out, FILENAME_MAX, stdin);
        if (out[0] == '\n')
        {
            strcpy(out, default_string);
            return;
        }
        out[strcspn(out, "\n")] = '\0';
    }

```

Код файла io.c

```

#include "io.h"
#ifdef _WIN32
#include <windows.h>
#else
#include <stdio_ext.h>
#include <termios.h>
#include <unistd.h>
#endif

// Открытие файла с обработкой ошибок
FILE* open_file(char *filename, bool binary, bool write_mode, bool
exit_on_fail)
{
    FILE* file = fopen(filename, (binary ? (write_mode ? "wb" :
"rb") : (write_mode ? "w" : "r")));

    if (file == NULL)
    {
        printf(SET_RED "Cannot open file %s!\n" RESET, filename);
        if (exit_on_fail) exit(0);
    }

    return file;
}

```



```

}

// Ввод целого числа в заданном диапазоне
int read_int_range(int from, int to)
{
    bool first_iteration = true;
    int x;
    do
    {
        if (!first_iteration) printf("Invalid format! Enter a number
(from %d to %d): ", from, to);
        x = read_int(stdin, false);
        first_iteration = false;
    } while (x < from || x > to);
    return x;
}

// Ввод целого числа
int read_int(FILE* file, bool binary)
{
    flush_buffer(file, true);
    int x;
    if (binary)
        fread(&x, sizeof(int), 1, file);
    else
        while (fscanf(file, "%d", &x) == 0)
        {
            printf(SET_RED "Invalid input! Expected an integer: "
RESET);
            flush_buffer(file, true);
        }
    return x;
}

// Ввод вещественного числа
float read_float(FILE* file, bool binary)
{
    flush_buffer(file, true);
    float x;
    if (binary)
        fread(&x, sizeof(float), 1, file);
    else
        while (fscanf(file, "%f", &x) == 0)
        {
            printf(SET_RED "Invalid input! Expected a float: " RESET);
            flush_buffer(file, true);
        }
    return x;
}

// Ввод строки
void read_string(char* str, FILE* file, bool until_ln)
{

```

```

    flush_buffer(file, true);
    fscanf(file, (until_ln ? "%[^\n]" : "%s"), str);
}

// Очистка буфера ввода
void flush_buffer(FILE* file, bool stdin_only)
{
    if (!stdin_only || file == stdin)
    {
#ifdef _WIN32
        rewind(file);
#else
        __fpurge(file);
#endif
    }
}

// Вывод символа n раз
void print_n_times(int n, char c)
{
    for (int i = 0; i < n; i++)
        printf("%c", c);
}

// Очистка экрана
void clear_screen()
{
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

```

ПРИЛОЖЕНИЕ Г
(Обязательное)

Ведомость документов

Обозначение		Наименование		Примечание	
		Графические документы			
ГУИР.6-05-0611-05.141 ПД1		Схема алгоритма функции main		А4	
ГУИР.6-05-0611-05.141 ПД2		Схема алгоритма функции tree_add_element		А4	
		Текстовые документы			
БГУИР.6-05-0611-05.141 ПЗ		Отчет по учебной (ознакомительной) практике		36 с.	